

Slide 1 — Project Overview

Flask E-Commerce Application

A server-rendered e-commerce platform demonstrating **full-stack web development** and **cloud computing concepts**.

What it does - Complete storefront: browse → review → cart → checkout → order management - Admin back office: products, users, orders, complaints - Runs behind a custom **round-robin load balancer** across multiple live instances

Why it is built this way - Server-rendered (Jinja2), so every page load is a real HTTP request that can be watched traversing the load balancer to a specific backend - Security treated as a first-class concern from the first commit, not retrofitted

By the numbers

Data models	7
Route blueprints	7
Automated tests	131 passing
Cross-browser checks	99/99 passing
Development span	~5 months

Slide 2 — Application Features

Core Functionality

User management - Registration and login with PBKDF2-SHA256 password hashing - Signed, expiring password-reset tokens (single-use) - Role-based access control via `is_admin` + `@admin_required`

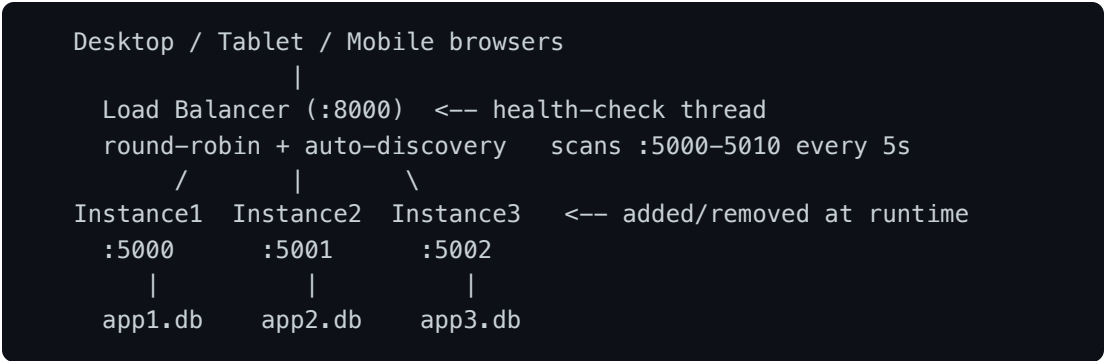
Catalog and reviews - Product listing, detail pages, many-to-many tag filtering - JSON API (`GET/POST/PUT/DELETE /products`), admin-gated for writes - Star ratings with a **database-level** one-review-per-user constraint

Cart and checkout - Session-backed cart with live badge, quantity updates, removal - Mock checkout — **no real payment processed, no card data stored** - Validation on **both** sides: HTML5 in the browser, re-validated on the server

Orders - History, detail, Buy Again, return requests, complaints - **Ownership-enforced**: another user's order returns **403 Forbidden**

Slide 3 — System Architecture

Four-Tier Design



Stack: Python 3.12 · Flask 3.x · SQLAlchemy · Flask-Login · Jinja2 · pytest · Docker · GitHub Actions · Codio

Load balancer implements three cloud concepts

Concept	Implementation
Round-robin	<code>itertools.count()</code> cycles the healthy pool
Health checking	Daemon thread probes every 5s, 2s timeout
Auto-scaling	Scans a port range — instances join/leave automatically

Hop-by-hop headers stripped per RFC 7230. Zero healthy backends → **503**, not a crash.

Slide 4 — Challenges and Solutions

Problems Worked Through

Challenge	Resolution
Flat <code>app.py</code> could not be tested	Restructured into <code>create_app()</code> factory accepting a test config — this is what made 131 tests possible
<code>psycpg2-binary</code> broke the Docker build	Split requirements; image installs a SQLite-only file
Docker rebuilt deps on every source edit	Copy requirements → install → *then* copy source
Static backend list showed no elasticity	Port-range scanning + health checks for true dynamic membership
SQLite locked under multi-instance writes	One database file per instance — a documented trade-off, not a hidden bug

Found and fixed during final testing

Issue	Impact	Fix
Nav overflowed on phones — "Logout" clipped off-screen	Control unreachable	<code>flex-wrap</code> + media query
Forms 5px wider than viewport	Horizontal scroll	Global <code>box-sizing: border-box</code>
Catalog took 5.8s on Slow 3G	Perceived performance	Lazy-loaded thumbnails → 968ms (83% faster)

Slide 5 — Testing and Results

Four Levels of Verification

1. Automated suite — 131 pytest tests passing In-memory SQLite fixtures; runs in CI on every push and PR alongside a Docker build.

2. Cross-browser — 99/99 checks passing

Engine	Represents	Purchase flow	Validation	403 gate	Responsive
Chromium	Chrome, Edge	Pass	Pass	Pass	Pass
Firefox	Firefox	Pass	Pass	Pass	Pass
WebKit	Safari	Pass	Pass	Pass	Pass

3. Responsive — zero overflow across 12 pages × 3 widths Measured programmatically (`scrollWidth` vs viewport), not judged by eye — which is how the nav clipping was caught when a screenshot had masked it.

4. Infrastructure — load balancer verified

Scenario	Result
Round-robin across 2 backends	Perfect alternation
Scale up — 3rd instance started at runtime	Joined rotation automatically
Scale down — instance killed	Evicted from rotation
Total failure — all backends down	HTTP 503 , no crash

Outcome

A functional, tested, documented e-commerce application that demonstrates horizontal scaling, health-checked load balancing, and auto-scaling behavior — with every issue found in final testing fixed and re-verified.

Repository: <https://github.com/tlycode/tanner-young-cloud-project>